Federated Identity

Computing Systems Security · MCS and MEI · University of Minho · 2026-27

Vítor Francisco Fonte <vff@di.uminho.pt> · 2026-08-19

Federated Identity: Lecture Overview

Cryptography as a Trust Instrument: what a signature proves, what a certificate binds, and what an authority is for

The Federation Problem: why the decision moves off the machine, and the shape these protocols share

OIDC in Depth: four actors, the authorization-code flow, and what is inside an ID token

Where It Breaks: the two misconfigurations, and why neither of them is broken cryptography

Into the Lab: a real provider, a real flow, and a verifier that only pretends to verify

The tab on the right edge of each slide shows the tier of the slide.

Lecture Core: covered in session. Lab Core: connects to laboratory work. Study Support: post-session review. Reference: lookup material.

What You Need From Cryptography

- You need what a signature **proves**, and to whom. Not how it is computed
- The mathematics is the Cryptography unit's subject. This half hour is its **use**
- Four objects, in order: a **keypair**, a **signature**, a **certificate**, an **authority**
- By the end you should be able to say what each one lets you conclude, and what it does not

If a slide here shows an exponentiation, something has gone wrong. This is a black box.

What a Signature Proves

- A signature binds **a message** to **a keyholder**. Those two things, and nothing else
- It gives you **integrity**: the message has not changed since it was signed
- It gives you **authenticity**: it was signed by whoever holds that private key
- It does **not** tell you who that keyholder is, whether they are honest, or whether the message is true
- Verification is a yes or no answer, and it is only as meaningful as your reason to care about that key

Integrity and authenticity, together, in one operation. Everything else you want must come from somewhere else.

A Private Key Acts as an Identity

- Two related keys. The **private** one signs and must stay with its holder; the **public** one verifies and travels
- Once a system binds the public key to you, holding the private key is what it means, operationally, to be you there
- So compromise of a signing key is, above all, a theft of identity
- Everything later today reduces to one question: **whose public key am I verifying against, and why that one?**

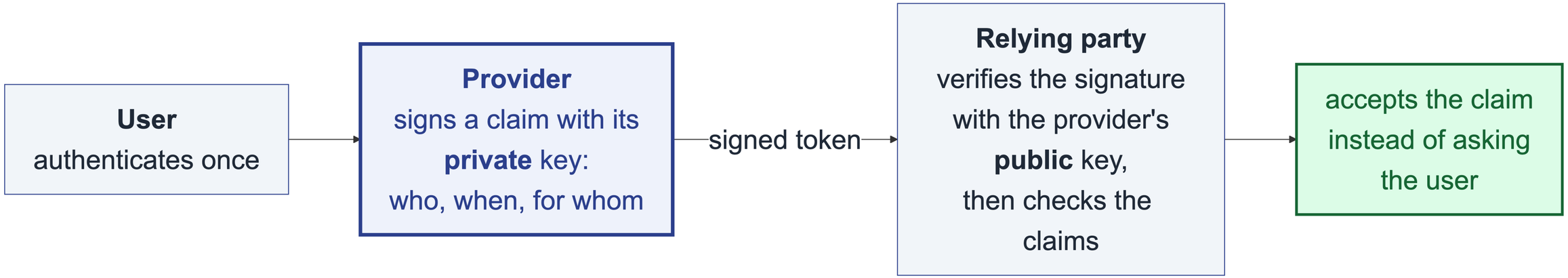
A stolen signing key lets somebody else be you, until verifiers stop trusting that key.

Certificates and Authorities



A certificate is testimony. The chain of who vouched ends at a trust anchor, a key you trust only because you installed it.

What a Token Signature Proves



Federation is delegation. Every guarantee you gain is the provider's guarantee, not yours.

Verifying Is How You Trust

- One idea carries into the lab: **verifying is how you trust something you did not create**
- Verifying means the signature, against a key you have reason to trust, and the claims your decision rests on
- Decoding is not verifying. Reading a token tells you what it says, not that anybody stood behind it
- Every step you skip is a step an attacker no longer has to defeat
- The laboratory's second failure is exactly this, and it is two lines of code

This sentence is the whole of the crypto half. If only one thing survives the hour, make it this one.

The Federation Problem

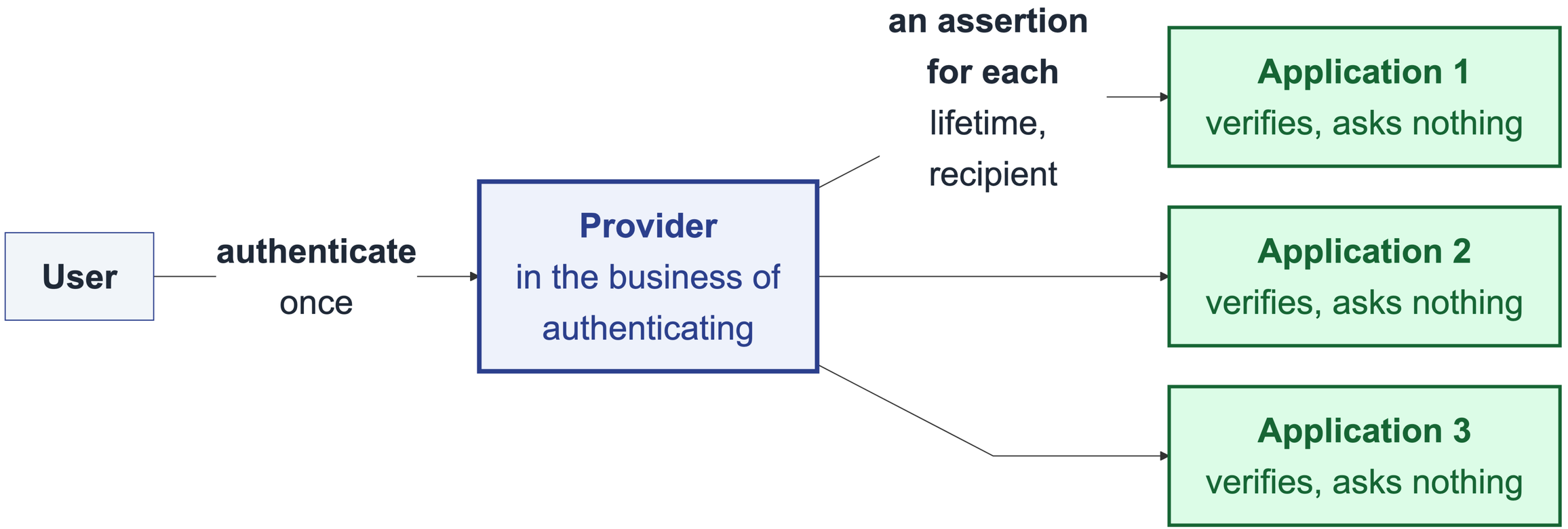
One user, forty applications. Where should the password live, and who should check it?

Why Not a Password per Application

- **Sprawl.** Each application stores credentials, so each one is a place to breach and a place to get storage wrong
- **No single revocation.** Somebody leaves and you are chasing forty systems, hoping the list is complete
- **Phishing surface.** Forty login pages teach users that any login page is normal
- Users answer sprawl by reusing one password everywhere, which turns forty independent risks into one

The problem is not that passwords are weak. It is that a password per application scales the wrong way.

The Shape of a Federation



Authenticate once, assert, verify. Learn this shape and the protocols in this family read as dialects of it.

Three Generations, One Idea

- **Kerberos.** Tickets, symmetric keys, within a realm or across trusted ones. Still what a Windows domain runs on
- **SAML.** XML assertions for enterprise single sign-on, browser-mediated, still everywhere in institutions
- **OAuth 2.0 and OIDC.** Tokens for web and mobile, JSON ID tokens, the layer under most 'log in with' buttons
- Different decades, different formats, the same three steps you just saw

Which one you meet is a question of era and ecosystem, not of which one is correct.

Study Support

Authentication or Authorisation

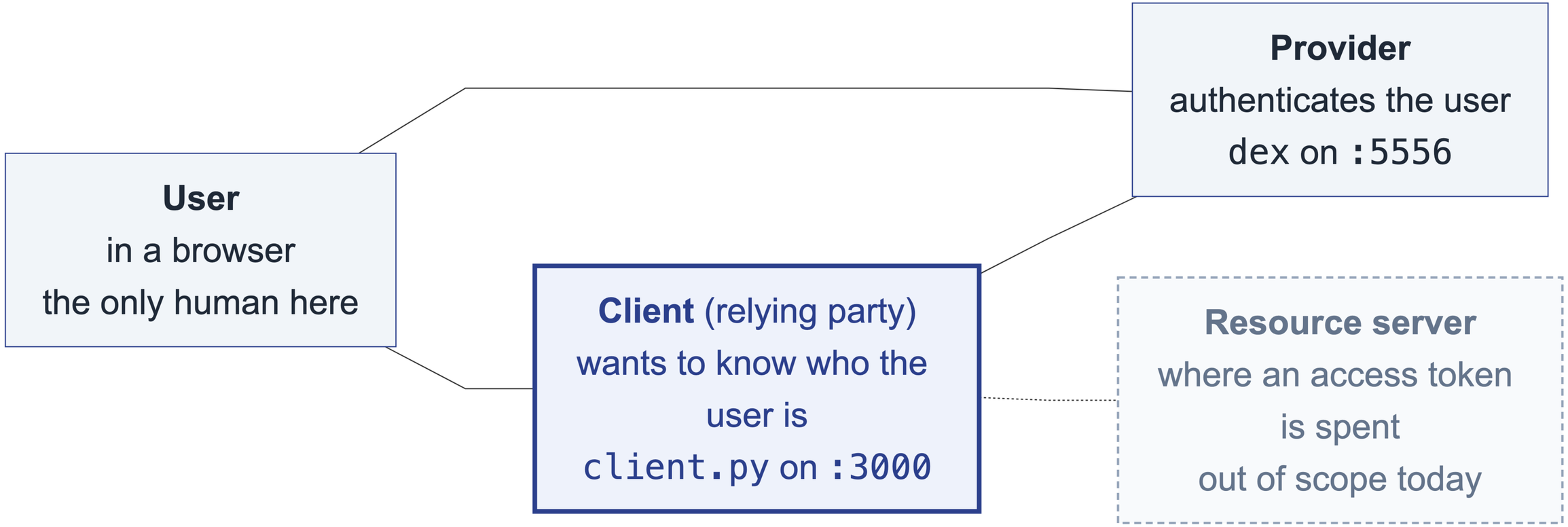
- **OAuth 2.0** was designed for **delegated access**: let this application act on my behalf, within these scopes
- **OIDC** adds the missing question: **who is the user**, answered by an **ID token** the client verifies
- The **access token** is for calling an API. It is not proof of identity, and its contents are not addressed to you
- Using an access token as a login is among the commonest real-world confusions in this family

The ID token says who; the access token says what may be done. Swapping them is a vulnerability, not a style choice.

OIDC in Depth

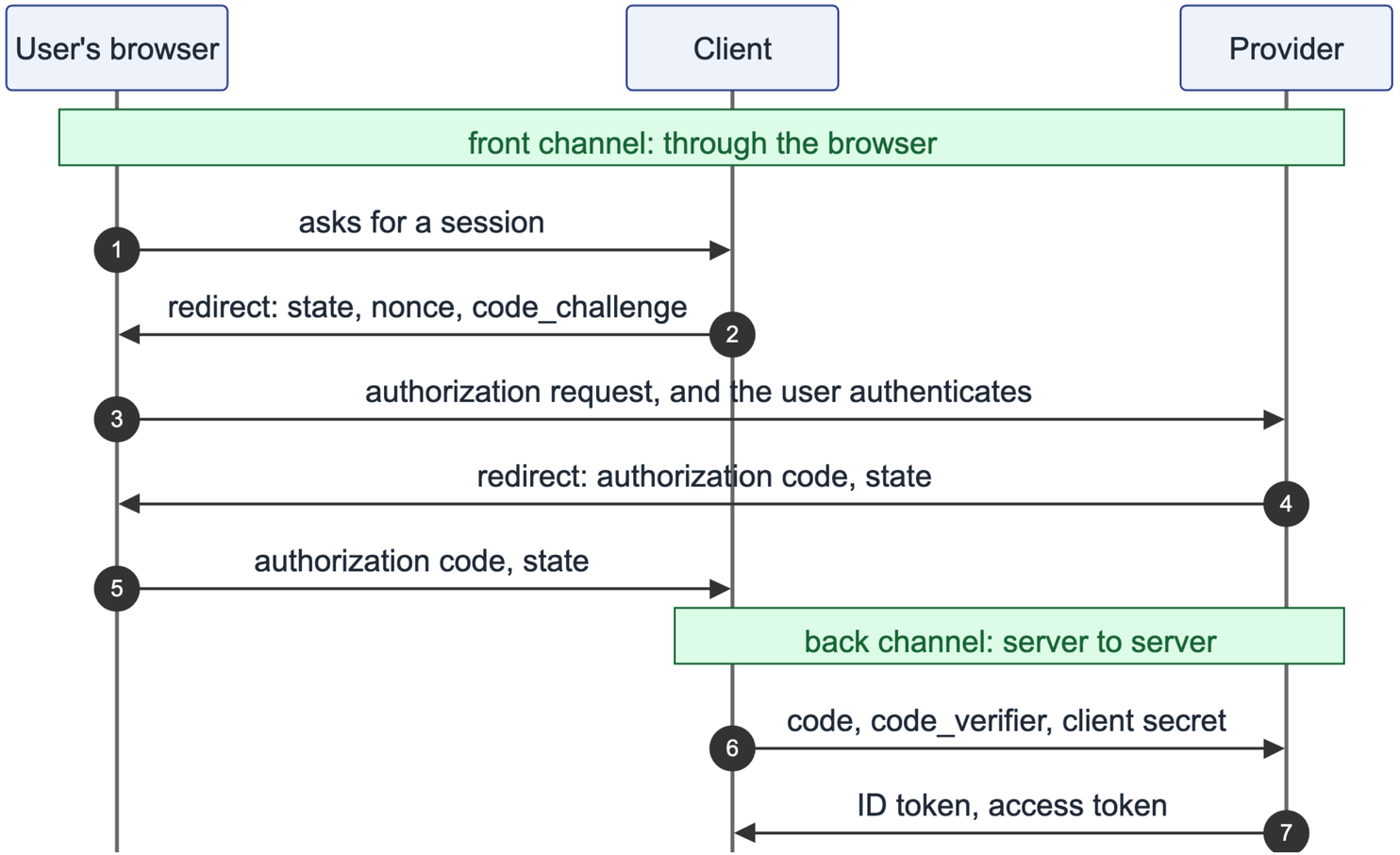
Four actors, seven messages, and two of them deliberately do not go through the browser. Why?

Four Actors



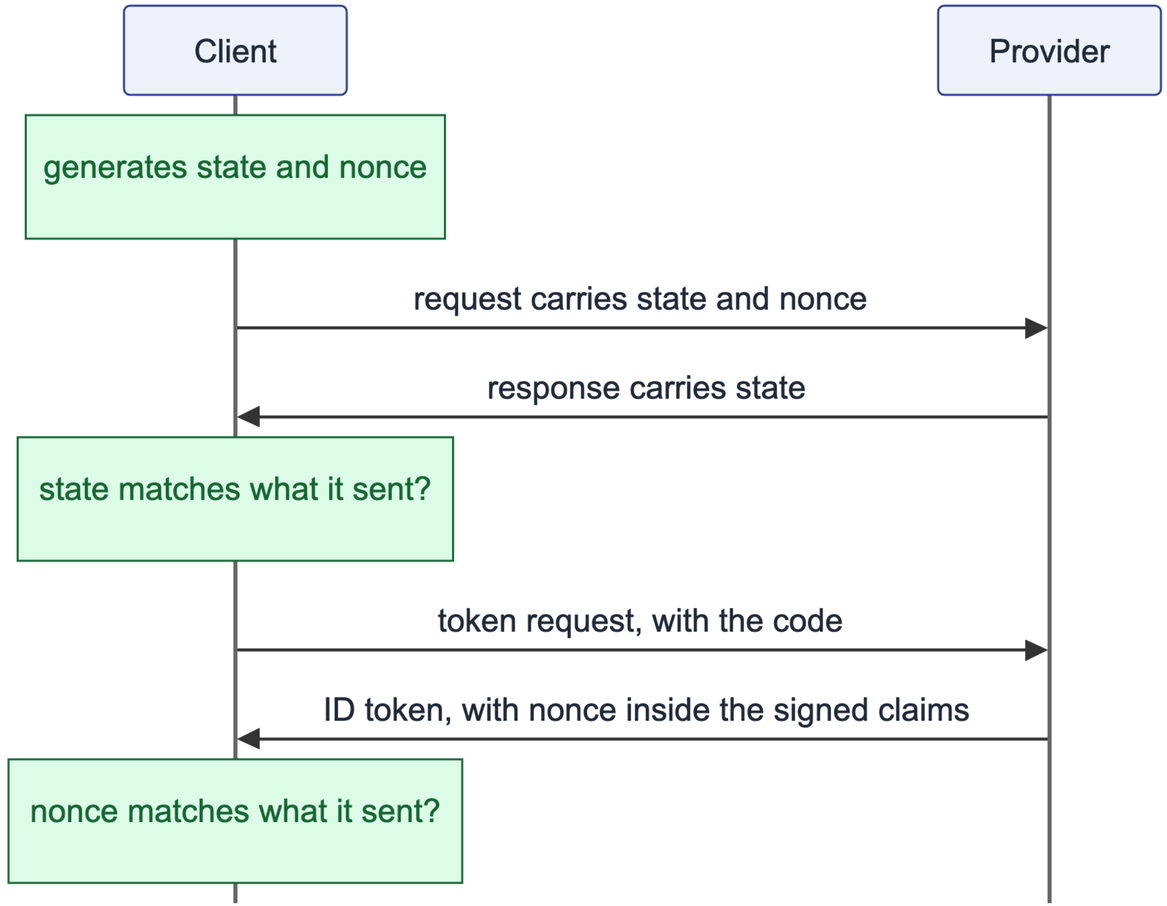
Name the actor before you name the message. Most confusion in this protocol is about who is talking.

The Authorization-Code Flow



The browser carries the redirects and the code. The token exchange never passes through it. In the lab you watch both.

state and nonce



state is checked on the response, nonce inside the token. Different jobs, each made for one flow and accepted once.

Inside the ID Token

```
header  { "alg": "RS256", "kid": "c8f3..." }

payload { "iss": "http://localhost:5556/dex",
          "aud": "democlient",
          "sub": "9e1a4c22-...-0004",
          "email": "student@ssc.local",
          "nonce": "...", "exp": 1789... }

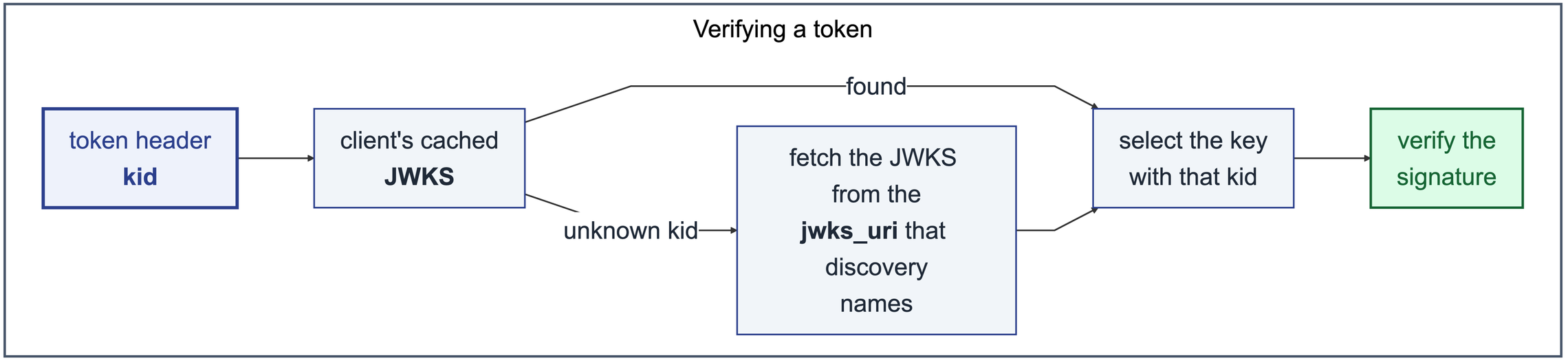
signature  <over header and payload>
```

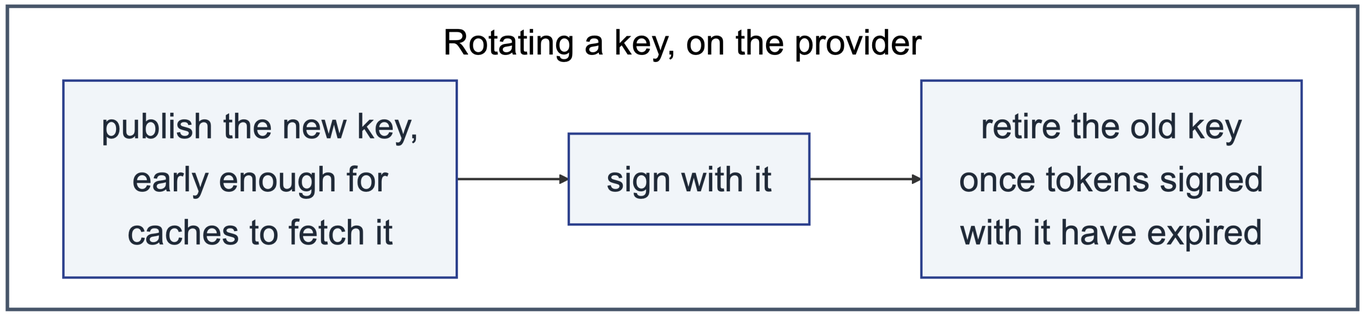
- **`iss`**: who issued it. Verify it is the provider you configured, not merely a provider
- **`aud`**: who it is for. If it is not you, it was not addressed to you, however valid it is
- **`exp`**: when it must stop being accepted. Freshness is a security property, not bookkeeping
- **`sub`**: the user's stable identifier at this issuer. Not the email, which can change hands
- **`nonce`**: the binding to your request, from the previous slide

Every claim you rely on and do not check is an attack somebody does not have to bother defeating.

Lab Core

JWKS and the Key Identifier





Rotation is why this indirection exists. A system that cannot change its keys cannot recover from losing one.

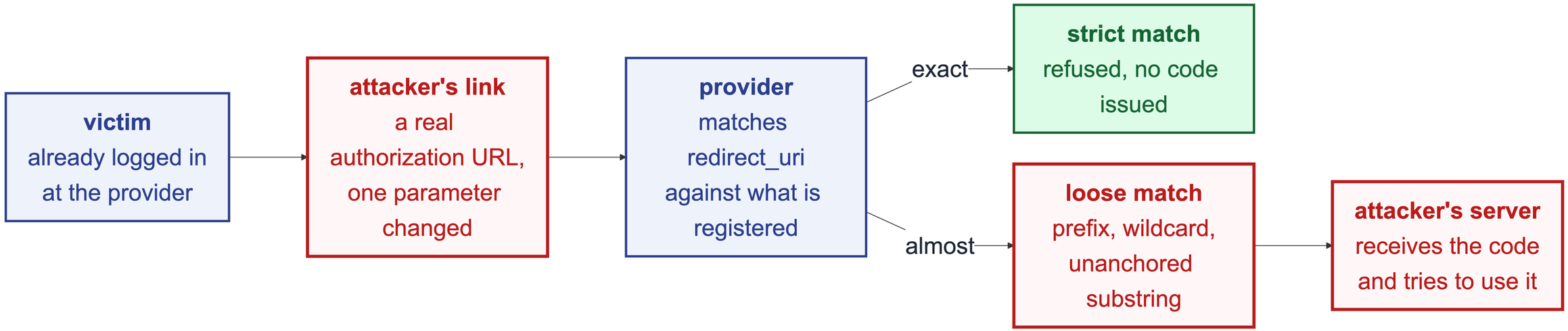
Study Support

Where It Breaks

Two real failure classes, neither of them a broken algorithm. Where would you look first?

Where the Code Ends Up

- **Correct:** exact match against the registered string. **Wrong but common:** prefix, wildcard subdomain, substring
- **Prefix rule:** anything that starts with `http://localhost:3000` passes, including `http://localhost:3000.attacker.example/callback`
- **PKCE** protects a code taken from somebody else's flow. It does not repair a loose matching rule



This is an implementation or configuration choice, not a protocol flaw. The protocol is fine; the matching rule is where it is lost.

Lab Core

Decoding Is Not Verifying

```
# verify.py, the way that holds
jwt.decode(token, key_from_jwks,
  algorithms=["RS256"],   # allow-list
  audience=AUDIENCE, issuer=ISSUER,
  options={"require": ["exp","iss","aud"]})

# verify_naive.py, the way breaches happen
json.loads(b64url_decode(token.split('.')[1]))
```

- The second fetches no key, checks no signature, validates no claim: a decoder called a verifier
- Edit any claim by hand and it still accepts, because there was never anything to fail
- Note the allow-list: it is what refuses a token whose header says `alg: none`
- Never let the token choose how it is verified. That is the attacker's field, not yours

Four lines apart. One of them is the classic breach, and it is the one that looks simpler.

Where the Vulnerabilities Live

- Not in the algorithms. RS256 is not what fails in these incidents
- They live in **validation omissions**: a claim not checked, a signature not verified, an algorithm not pinned
- And in **configuration**: a matching rule too loose, a secret shared too widely, a token accepted from the wrong party
- Which is good news, because both are things you can read off a configuration file and a code review

Assume the algorithm works and audit everything around it, including which algorithm you accept. That is where the findings are.

Today in the Laboratory

The whole stack runs inside your VM: no external service, no real account, no credential of yours anywhere in it.

- Bring up `dex` and the demo client, and run a **real** authorization-code flow in the browser
- Decode the ID token you were issued, and find every claim in it that this hour named
- Tamper with the `redirect_uri` and watch a correct provider refuse you
- Feed a hand-edited token to both verifiers, and see one refuse it and the other say thank you

You will see one defence hold and one verifier fail. Write down which of the two you would have shipped.

Lab Core

One Question, One Layer Down

- Today moved the question off the machine: the provider authenticates, the application verifies its statement
- Next week does the same move for **host** access, with an SSH certificate authority
- Same three steps, a trust anchor of its own, a much smaller protocol, and a different revocation problem
- Then the block's report asks you to compare all three answers, including Week 3's PAM

The machine, a provider, an authority. Three answers to one question, and each fails in its own way.

Reading and Reference, All Optional

- **OpenID Connect Core 1.0**. The specification. Section 3.1 is the authorization-code flow; Section 3.1.3.7 is the ID token validation list.
  https://openid.net/specs/openid-connect-core-1_0.html
- **RFC 6749 and RFC 9700**. OAuth 2.0 itself, and its current security best practice. The second is the more useful read for this unit.
  https://www.rfc-editor.org/rfc/rfc9700
- **RFC 7636, Proof Key for Code Exchange**. PKCE, in nine pages: why a leaked code is not enough without its flow's verifier.
  https://www.rfc-editor.org/rfc/rfc7636
- **Stallings and Brown, Computer Security: Principles and Practice, 4th ed., 2018**. Chapter 4 for Kerberos and the federated identity background.
- **dex documentation**. The provider used in the laboratory. Its configuration file is the whole of its setup.
  https://dexidp.io/docs/

Reference

Key Takeaways

- A signature binds a message to a keyholder. It says nothing about who that holder is, or whether they are honest
- A trust anchor is where the regress stops: trusted by configuration, and therefore worth attacking
- Federation is delegation. The guarantees you gain are the provider's; checking them is still yours
- The ID token says who; the access token says what. Confusing them is a vulnerability
- **Verifying is how you trust what you did not create**: signature, key and claims. Decoding is not verifying

Assume the algorithm works. The findings are in what was not checked, which algorithm was accepted, and what was configured loosely.